Abstracting Realistic VR Avatars in Unity Using Toon Shading and Edge Detection: A Practical Report

Nidanur Gunay
University of Konstanz
Germany
nidanur.guenay@uni-konstanz.de

Abstract

Virtual reality applications increasingly rely on avatar representations to facilitate social presence and user embodiment. However, photorealistic avatar rendering can trigger the *uncanny valley* effect, where near realistic but imperfect human representations evoke discomfort rather than engagement. Non-photorealistic rendering (NPR) offers an alternative approach, using stylization to abstract away from photorealism while maintaining expressiveness.

This work presents an experimental comparison of four toon shading implementations for VR avatar rendering in Unity's Universal Render Pipeline. The techniques investigated include: (1) quantized diffuse lighting with inverted hull silhouette outlines, (2) screen-space normal edge detection using hardware derivatives, (3) texture-based Sobel edge detection without pre-filtering, and (4) Gaussian pre-filtered Sobel detection with multi-pass smoothstep sharpening.

Testing revealed that world-space geometry outlines provided stable avatar silhouettes essential for VR applications, whereas screen-space normal derivatives exhibited significant distance dependent artifacts. Furthermore, whilst Gaussian pre-filtering is theoretically recommended to suppress noise, results suggest that it primarily softened inner detail lines rather than producing a discernible improvement over the unfiltered Sobel implementation. The documented trade-offs between geometric stability, texture-based detail, and noise suppression offer practical guidance for future NPR shader development in VR contexts.

1 Introduction

The rapid growth of virtual reality platforms has created unprecedented demand for effective avatar representation systems. Research demonstrates that avatar appearance significantly affects perceived social presence [13] and interpersonal trust in virtual environments [3]. This relationship suggests that the visual design of a character is not merely aesthetic but a functional component of user embodiment, directly influencing how users perceive and interact with one another in digital spaces.

Conventional approaches to avatar rendering have pursued photorealism by leveraging advances in physically-based rendering, subsurface scattering, and high-resolution texture mapping. However, this pursuit of realism encounters a fundamental perceptual obstacle: the uncanny valley. First described by Mori in 1970, the uncanny valley hypothesis proposes that near realistic human representations elicit unpleasantness and revulsion [12]. Concerning this effect, non-photorealistic rendering (NPR) offers a robust alternative approach. Unlike rendering methods that strive for accurate simulation of light and materials, NPR techniques deliberately stylize the rendering output to achieve artistic or communicative goals [8]. Thus, NPR serves as a tool to abstract avatar fidelity and mitigate the negative uncanny valley effect.

One of the prominent branches of NPR is toon shading, a stylization technique that simplifies the appearance of 3D models by grouping shading values into discrete bands, producing a flat, illustrative aesthetic. This approach effectively translates complex three-dimensional volumes into distinct regions of color by substituting the smooth gradients of traditional Lambertian reflectance with a quantized stepped function.

Complementary to quantized shading, edge or contour detection serves as a fundamental component of the NPR pipeline by accentuating object boundaries and enhancing visual legibility in abstracted scenes. A wide range of techniques have been developed to achieve this objective, each offering distinct trade-offs. For instance, while screen-space strategies offer high efficiency by operating on the rendered buffer, they often lack the temporal and distance-based stability required for immersive VR environments. To address this, many stylized pipelines utilize the inverted hull (or geometry expansion) method to generate robust silhouette outlines.

This inverted hull technique involves a dedicated rendering pass where the mesh is slightly expanded along its vertex normals and rendered with back-face culling, creating a secondary shell that serves as a consistent outer boundary. As established by Lake et al., this hardware-accelerated approach is often preferred for real-time applications because it provides a simple, scalable solution that avoids the computational overhead of complex mesh pre-processing or CPU side calculations [10]. By combining these geometric outlines with the illustrative detail extracted via Sobel filters, it is possible to achieve a multilayered abstraction that effectively preserves the expressive form of a character while firmly mitigating the uncanny valley effect.

This report presents a practical exploration of creating abstracted avatars within the Unity engine through the development of customized shading techniques. By iteratively implementing toon shading and edge detection methods, the project investigates how varying levels of stylization can be used to navigate the perceptual constraints of virtual reality.

2 Related Work

NPR has extensively explored stylized shading and contour extraction as alternatives to photorealistic depiction. Early work by Gooch et al. [8] demonstrated how modified illumination models could enhance shape perception for technical illustration, laying conceptual groundwork for toon shading, also known as cel shading. These approaches replace continuous Lambertian reflectance with

discretized lighting bands, producing a flattened and stylized visual appearance that emphasizes structural form over photometric realism.

Furthermore, contour and silhouette rendering constitute a significant component of NPR pipelines. Geometry-based techniques, particularly the inverted hull or shell method, generate outlines by expanding back-facing polygons along their normals. Lake et al. [10] showed that this strategy is well suited for real-time animation due to its simplicity and predictable performance characteristics. A later broader survey of silhouette extraction algorithms by Isenberg et al. [9] categorizes geometry-based methods as robust and distance independent but inherently limited to object boundaries rather than internal surface detail. Thus, these geometric methods are not capable of identifying structural boundaries and cannot convey internal surface details.

Another efficient alternative to capture geometric features beyond silhouettes is screen-space derivative techniques. Modern GPUs process fragments in 2×2 pixel groups, enabling hardware-accelerated computation of partial derivatives via the `ddx()` and `ddy()` intrinsics [2]. By computing these derivatives of interpolated surface normals which manifest as gradients, discontinuities in surface orientation such as creases, mesh boundaries, and curvature changes can be detected without pre-processing or additional render passes. While computationally efficient, screen-space techniques exhibit inherent distance dependence: edge thickness is measured in pixels rather than world units, potentially causing artifacts when viewing distance varies.

Consequently, image processing techniques have advanced to facilitate more precise inner line detection through pixel-level analysis. A cornerstone of this approach is the Sobel operator, a discrete differentiation tool widely utilized in computer vision and NPR to approximate image intensity gradients. By identifying regions of high spatial frequency, the Sobel filter allows for the integration of illustrative detail directly into the existing shading pipeline without the overhead of additional geometry passes. However, the mathematical process of differentiation inherently amplifies noise at high frequencies, which can lead to visual artifacts such as spurious or unstable lines. To address this sensitivity, seminal research by John Canny and Marr and Hildreth suggests that a form of low pass smoothing, typically via a Gaussian filter, is essential prior to differentiation to ensure structural stability [4, 11]. By incorporating these classical signal processing principles into the rendering pipeline, the shader can suppress spurious gradients while preserving the meaningful intensity transitions that define a character's form.

3 Methodology

3.1 Avatar Model and Animation

The avatar used throughout this project was the humanoid character model named 'Jody'. It was obtained from Adobe Mixamo, a web-based service for 3D character rigging and animation [1]. The Jody model was specifically selected for its realistic appearance and standardized humanoid skeleton structure, ensuring compatibility with Unity's animation system. This compatibility provided a reliable baseline for testing NPR shaders on realistic proportions. Furthermore, the efficient integration of various animations from the Adobe Mixamo library allowed for a dynamic assessment of the shader's performance.

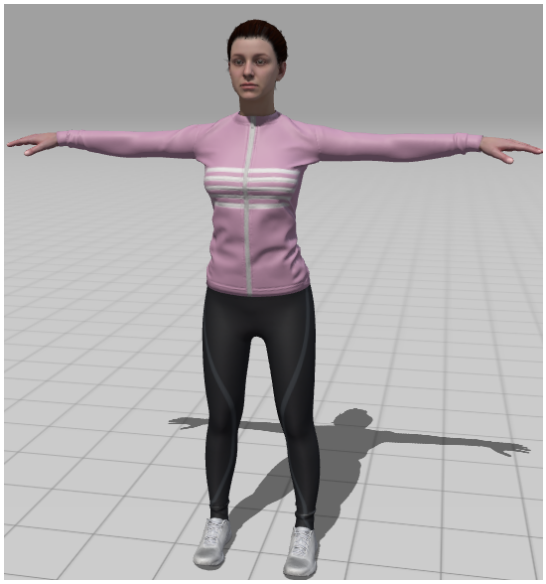

Figure 1: Realistic Humanoid Avatar Jody for Shader Experiments [1]

3.2 Development Environment

The project was developed in Unity version 6 (6000.0.60f1), a widely adopted real-time engine for interactive applications such as games, simulations, and virtual reality experiences [15].

Rendering was handled by the Universal Render Pipeline (URP), one of Unity's Scriptable Render Pipelines (SRP) to balance visual quality with performance across a broad range of hardware [18]. This architecture provided the essential technical flexibility to develop and iterate upon customized shaders for the various experimental NPR techniques applied to the VR avatars.

3.3 Shader Architecture

Unity shaders follow a hierarchical structure in which a `Shader` contains one or more `SubShaders`, each containing various `Passes` [16].

- **Properties block**: Exposes adjustable parameters such as colors, textures, toggles, sliders etc. within the Unity Inspector, enabling shader customization without recompilation.
- **SubShader block**: Unity selects the first compatible SubShader for current hardware [17], and it defines the rendering configuration for a specific pipeline.
- **Pass blocks**: Each Pass represents a single draw call through the GPU pipeline and has the ability to define its own shader programs, blending operations, depth testing behavior, and culling mode. This employs a flexible strategy to integrate multiple stylization techniques within a single shader material. For instance, toon shader employs two passes:

(1) **Outline Pass**: Renders back-faces with front-face culling while expanding vertices along normals to create silhouette outlines.
(2) **ForwardLit Pass**: Renders front-faces with toon shading, edge detection, and rim lighting.

3.4 Iterative Development Process

The shader pipeline was developed iteratively across several versions, each introducing additional techniques while preserving the functionality of earlier iterations. Version 1 established the baseline toon shading and inverted hull outline rendering. Version 2 introduced screen-space normal-derivative edge detection to investigate geometry-based inner line detection. Version 3 added Sobel-based texture edge detection, and Version 4 incorporated Gaussian pre-filtering for noise suppression. Each version was tested on the animated Jody avatar under varying lighting conditions and camera distances to assess visual quality, temporal stability, and performance impact.

4 Results

The project aimed to transition the 'Jody' model from a realistic aesthetic toward an abstracted representation while preserving structural integrity. This iterative process enabled a comparison of the effects of various NPR techniques, ranging from geometry-based outlines to screen-space filters, on the final visual coherence within a real-time rendering context.

4.1 Version 1: Basic Toon Shading with Geometry-Based Outlines

The initial shader version established the technical baseline for subsequent experimental versions by implementing two core NPR techniques: quantized diffuse shading (toon shading) and silhouette outlines generated through geometry expansion, a technique commonly employed in real-time cartoon rendering [9].

4.1.1 Methods and Implementations.

- **Quantized Diffuse Lighting (Toon Shading)**
  The toon shading model replaces the continuous Lambertian diffuse term with a discrete, stepped function. In traditional diffuse shading, surface brightness varies smoothly in accordance with $N \cdot L$, which is the dot product of the light direction $\mathbf{L}$ and the surface normal $\mathbf{N}$ [8]. This formulation produces smooth gradients that effectively convey 3D form. However, it results in a photorealistic rather than stylized visual appearance. In contrast, toon shading first computes the continuous lighting value per pixel and subsequently partitions it into a limited number of discrete levels. This quantization produces the characteristic flat visual style, where shadows and highlights appear as distinct bands of color rather than smoothly blended gradients [10]. The implementation achieves this effect using the floor function:

$$t = \frac{\lfloor s \cdot n \rfloor}{n} \tag{1}$$

  where s denotes the smoothed lighting value and n represents the number of discrete shading bands. The floor operation truncates the continuous lighting value into discrete intervals, producing the banded appearance characteristic of traditional hand-drawn animation.
- **Smoothstep Shadow Transition**
  Prior to the quantization stage, a smoothstep function is used to control the position and sharpness of the shadow boundary. In other words, the smoothstep function performs smooth Hermite interpolation between 0 and 1, which introduces a smooth transition instead of an edge. This modification enhances the visual quality of the avatar, since abrupt lighting effects are avoided.
  In the shader implementation, the function is applied around a configurable threshold τ with smoothness parameter σ, and is defined by the following expression:

$$\text{smoothstep}(\tau - \sigma, \tau + \sigma, \mathbf{N} \cdot \mathbf{L}) \tag{2}$$

  Lower values of τ yield sharper transitions, almost resembling the step function. Moreover, greater values result in smoother gradients which can be used to reduce temporal aliasing effects in animations. Consequently, this approach provides an adjustable transition zone between illuminated and shadowed regions, allowing fine control over the stylized appearance of the shading model.
- **Shadow Strength Modulation**
  The contrast between illuminated and shadowed regions is controlled by a shadow strength parameter α that interpolates between the quantized toon value and full brightness:

$$t' = \text{lerp}(1.0, t, \alpha) \tag{3}$$

  The surface appears uniformly lit regardless of orientation when $\alpha = 0$ since the interpolation returns a constant value of 1.0. In contrast, the full quantized shading result is applied when $\alpha = 1$. Intermediate values of α provide continuous control over shadow intensity, enabling fine adjustment of shadow contrast while preserving the characteristic stepped aesthetic of toon shading.
- **Inverted Hull Silhouette Outlines**
  As Isenberg et al. detailed in their survey of silhouette algorithms, a common hybrid technique for rendering outlines involves the enlargement of back-facing polygons [9]. This technique, commonly referred to as the inverted hull or shell method, exploits GPU face culling to produce outlines without requiring edge detection or post-processing.
  This method operates using two rendering passes:
  (1) **Outline Pass:** The mesh is rendered with front-face culling, displacing each vertex outward along its normal by a fixed distance w. The displaced position $\mathbf{P}'$ is defined as:

$$\mathbf{P}' = \mathbf{P} + \mathbf{N} \cdot w \tag{4}$$

  Rendering only the back-facing geometry in this expanded state produces a shell slightly larger than the original model.
  (2) **Main Pass:** The mesh is subsequently rendered using standard back-face culling. The front-facing polygons overwrite the interior of the previously rendered shell, leaving only the protruding rim visible as a silhouette.

Lake et al. establish that this robust approach is a preferred solution for interactive animation and real-time NPR because it provides a simple and scalable solution [10]. By leveraging the standard graphics pipeline's ability to perform back-face culling and z-buffering, the technique maintains predictable performance characteristics even with complex topologies and meshes. Furthermore, because the expansion occurs entirely on the GPU during the vertex transformation stage, it eliminates the need for expensive mesh pre-processing or CPU-side silhouette calculations.

- **Fresnel Rim Lighting**
  The Fresnel effect describes the physical phenomenon in which a surface's specular reflectance varies as a function of the incident angle, increasing significantly as the viewing direction approaches grazing incidence. For computational efficiency, Schlick [14] proposed an efficient approximation of the Fresnel term:

$$F(\theta) = F_0 + (1 - F_0)(1 - \cos\theta)^5 \tag{5}$$

  where F_0 is the reflectance at normal incidence and θ is the angle between the view direction and surface normal.
  In the present implementation, a simplified variant inspired by Schlick's formulation is employed to produce a stylized rim lighting effect:

$$r = (1 - \text{saturate}(\mathbf{V} \cdot \mathbf{N}))^p \tag{6}$$

  where saturate clamps its argument to the range $[0, 1]$, $\mathbf{V}$ is the view direction, $\mathbf{N}$ the surface normal, and p a user-controlled exponent. Unlike Schlick's physically derived exponent of five, the parameter p is treated here as a stylistic control variable.
- **Constant Ambient Lighting**
  A constant ambient term is incorporated into the final lighting calculation to prevent fully shadowed regions from appearing completely black, ensuring that the character remains visible regardless of light orientation. The ambient contribution is independent of surface orientation and light direction and defined as:

$$L_{\text{total}} = L_{\text{light}} \cdot t' + L_{\text{ambient}} \tag{7}$$

  This formulation follows the simplified illumination model described by Foley et al., where ambient light is utilized as a global constant to simulate the effect of indirect light scattered from the environment [6]. The inclusion of L_{ambient} preserves visibility and form in shadowed areas, which is essential for maintaining visual clarity in real-time NPR.
- **Alpha Testing for Transparency**
  Avatar models often include semitransparent elements, such as eyelashes and hair cards rendered with alpha-tested textures. To handle these correctly, the shader implements alpha testing using the GPU's clip operation to discard fragments whose alpha values fall below a specified threshold α_{cutoff} [19]. Formally, this operation can be expressed as:

$$\text{if } \alpha_{\text{texture}} < \alpha_{\text{cutoff}} \text{ then discard} \tag{8}$$

  Alpha testing is executed in both the outline and main rendering passes to ensure silhouette consistency. Compared to alpha blending, this binary transparency approach avoids depth-sorting complexity and maintains real-time performance [19]. Without this operation, semitransparent geometry would produce undesired rectangular outlines rather than contours conforming to the visible texture region as shown in Figure 2.

Figure 2: Alpha Test Enabled/Disabled Jody

- **Depth Bias for Z-Fighting Prevention**
  The outline pass is susceptible to z-fighting artifacts, which manifest as flickering where the expanded shell geometry intersects with the main mesh at similar depth values. An optional depth bias shifts the outline geometry slightly along the view axis in clip space to mitigate these artifacts:

$$z'_{\text{clip}} = z_{\text{clip}} - \epsilon \cdot b \tag{9}$$

  As discussed by Akenine-Möller et al., this displacement is a standard technique used to resolve depth-testing ambiguities by ensuring one set of fragments is consistently favored by the depth buffer [2]. In the context of the inverted hull method, applying a small constant ϵ scaled by a bias strength b ensures that the outline pass is drawn with a slight depth offset relative to the main geometry.

4.1.2 Observations. Version 1 established the foundational toon shading pipeline with inverted hull silhouette outlines. The quantized diffuse lighting produced the characteristic stepped shading bands associated with cartoon animation, while the inverted hull technique provided clean, consistent silhouettes regardless of viewing distance. The Fresnel rim lighting enhanced character separation from the background by brightening edges that face perpendicular to the viewer.
However, testing revealed a significant limitation: silhouette outlines alone cannot convey internal surface details. Facial features such as eyes, nose contours, and lip boundaries remain unmarked. Similarly, clothing folds, hair strands, and decorative patterns painted into the diffuse texture produce no visible edge lines. This observation motivated the exploration of additional edge detection techniques in subsequent versions.

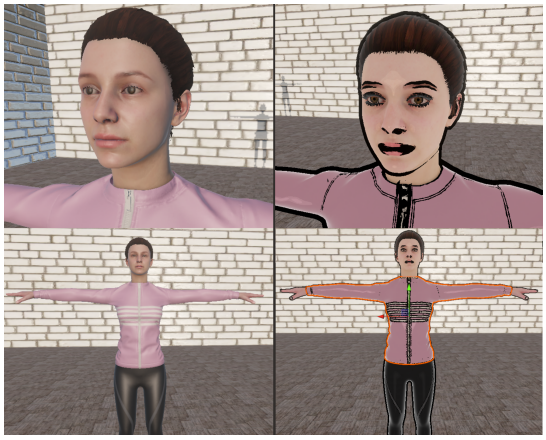

Figure 3: Comparison of Realistic Jody and V1

4.2 Version 2: Normal-Based Edge Detection Using Screen-Space Derivatives

This method was implemented experimentally to investigate the impact of screen-space normal edge detection on avatar rendering, comparing its behavior against texture-based (Sobel) and inverted hull silhouette outlines approaches.

4.2.1 Theoretical Background. The mathematical foundation for this approach lies in differential geometry. At points where the surface normal changes rapidly such as creases, corners, and mesh boundaries, the spatial derivatives of the normal vector will be large. By computing these derivatives and thresholding the result, geometric edges can be detected independently of texture content.

4.2.2 Why Screen-Space Instead of World-Space? A fundamental design decision in this implementation was to compute normal derivatives in **screen-space** rather than world-space. This choice was made because it leverages hardware-accelerated derivatives (ddx/ddy) at negligible cost, requires no mesh pre-processing, and automatically adapts to screen resolution. This enabled practical observations about the suitability of screen-space techniques for VR avatars.

4.2.3 Implementation. The shader computes the magnitude of normal variation across the screen and the gradient magnitude is computed as:

$$M_N = \sqrt{\left(\frac{\partial \mathbf{N}}{\partial x}\right)^2 + \left(\frac{\partial \mathbf{N}}{\partial y}\right)^2} = \sqrt{\|\mathbf{N}_x\|^2 + \|\mathbf{N}_y\|^2} \tag{10}$$

where $\mathbf{N}_x = \text{ddx}(\mathbf{N})$ and $\mathbf{N}_y = \text{ddy}(\mathbf{N})$ are the partial derivatives of the world-space normal with respect to screen coordinates. Following gradient magnitude computation, the shader applies a smooth thresholding function to classify pixels as edge or non-edge regions. The implementation uses a smoothstep interpolation rather than employing a binary step function, which would potentially produce aliased transitions.

$$e_N = \text{smoothstep}(\tau - \sigma, \tau + \sigma, M_N) \cdot \alpha_N \tag{11}$$

where τ is the detection threshold, σ controls the smoothness of the transition, and α_N is the edge strength coefficient. The resulting edge intensity $e_N \in [0, 1]$ is then composited with the lit surface color through linear interpolation:

$$\mathbf{C}_{\text{final}} = (1 - E)\,\mathbf{C}_{\text{shaded}} + E\,\mathbf{C}_{\text{edge}} \tag{12}$$

where C_{shaded} is the toon shaded surface color computed from the lighting pipeline and C_{edge} is the edge color. This compositing approach ensures that the detected edges smoothly blend with the underlying shading rather than producing hard discontinuities.

The entire normal edge detection pipeline executes within a single rendering pass alongside the lighting calculations. This pass shares the same foundational techniques established in Version 1: quantized diffuse lighting, smoothstep shadow transitions, shadow strength modulation, Fresnel rim lighting, constant ambient lighting and alpha testing for transparency. The normal-based edge detection augments this pipeline by introducing geometry-aware lines without requiring additional draw calls or intermediate render targets.

4.2.4 Observations. This stage of development introduced screen-space normal edge detection as an experimental approach to capture geometric features invisible to other implemented methods. Although the technique could successfully identify creases, mesh boundaries, and surface curvatures, significant artifacts emerged during distance testing. The screen-space edge detection produces increasingly thick edges that expand to cover substantial portions of the character's surface as the camera moves farther from the avatar. This observation led to the decision to retain inverted hull outlines for silhouettes in subsequent versions, using them for their world-space stability while exploring texture-based methods for internal detail lines.

Figure 4: Comparison of Realistic Jody and V2

4.3 Version 3: Texture-Based Edge Detection Using the Sobel Operator

To address the absence of inner detail lines identified in earlier versions, Version 3 introduced texture-based edge detection using the Sobel operator. This extension enabled the shader to extract detail

lines directly from the avatar's diffuse texture maps, enhancing the representation of fine visual features.

4.3.1 Implementation. The shader samples the diffuse texture at nine positions arranged in a 3×3 grid centered on the current fragment's UV coordinates. The spacing between samples is controlled by the `_InnerLineBlur` parameter, which scales a base offset value.

Larger offset values spread the sample points farther apart, effectively increasing the scale at which edges are detected. This can be useful for capturing broader features while suppressing fine texture noise, though at the cost of reduced spatial precision.

- **The Sobel Operator**
  The Sobel operator is a discrete differentiation operator used in image processing and computer vision for edge detection. It computes an approximation of the gradient of image intensity at each pixel, producing output that emphasizes regions of high spatial frequency which typically correspond to edges.
  As detailed by Gonzalez and Woods, the operator employs two 3×3 convolution kernels to estimate the horizontal and vertical components of the image gradient [7]. These kernels are defined as:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \quad G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \tag{13}$$

  The kernel G_x is designed to detect edges with a vertical orientation, corresponding to horizontal changes in intensity, whereas $G_y$is sensitive to edges with a horizontal orientation, corresponding to vertical changes in intensity. The central column or row of zeros ensures that the operator responds to local gradient variations rather than absolute intensity values.
  Each kernel was convolved with the image neighborhood centered at the pixel of interest. Because the Sobel operator requires scalar intensity values, the RGB texture samples were first converted to a scalar luminance (grayscale) representation prior to kernel application.
  The convolution operation yields the gradient on both horizontal and vertical direction. Furthermore, the magnitude of the gradient vector provides a scalar measure of edge strength at each pixel. According to the standard practice in digital image processing, the gradient magnitude M is computed as the Euclidean distance of the directional components [7]:

$$M = \sqrt{G_x^2 + G_y^2} \tag{14}$$

- **Texture Edge Detection**
  Higher magnitude values indicate stronger edges, that is, more rapid transitions in intensity. Hard thresholding was applied using the `step()` function to classify pixels as edge or non-edge based on whether the magnitude exceeds a user defined threshold. Pixels with gradient magnitude exceeding this threshold value were classified as edges while all others were discarded. The resulting binary mask was scaled by `_InnerLineStrength` parameter to allow artists to control the visual prominence of detected edges. In order to integrate these lines into the rendering pipeline, the final output color $\mathbf{C}_{\text{final}}$ was computed by blending the toon shaded color $\mathbf{C}_{\text{shaded}}$ with the edge color $\mathbf{C}_{\text{edge}}$ using the edge mask $E \in [0, 1]$.

$$C_{\text{final}} = (1 - E)\, C_{\text{shaded}} + E\, C_{\text{edge}} \tag{15}$$

This formulation allows for the direct integration of illustrative detail into the existing shading pipeline without the computational overhead of an additional rendering pass.

4.3.2 Observations. The third iteration implemented raw Sobel edge detection on the diffuse texture without any pre-filtering, serving as a baseline for comparison with filtered approaches. The Sobel operator successfully detected painted texture features including facial details, clothing patterns, and color boundaries. However, Version 3 exhibited high sensitivity to small parameter adjustments. The empirical evaluation of Version 3 identified 0.2 as the optimal threshold parameter for the Sobel operator, which was utilized to render the final stylized result shown in Figure 6. This specific value was selected after a systematic assessment of parameter sensitivity, the results of which are documented in the comparative analysis in Figure 5. As demonstrated in this comparison, the implementation exhibits extreme sensitivity; even marginal adjustments as small as 0.01 introduced significant artifacts on the avatar's face, a region of high perceptual importance. Furthermore, lowering the threshold resulted in the face becoming covered with spurious edge detections triggered by subtle skin texture variations while increasing the value led to disappearance of meaningful edge details. This instability motivated the introduction of Gaussian pre-filtering in Version 4.

Figure 5: Comparison of Inner Line Threshold values

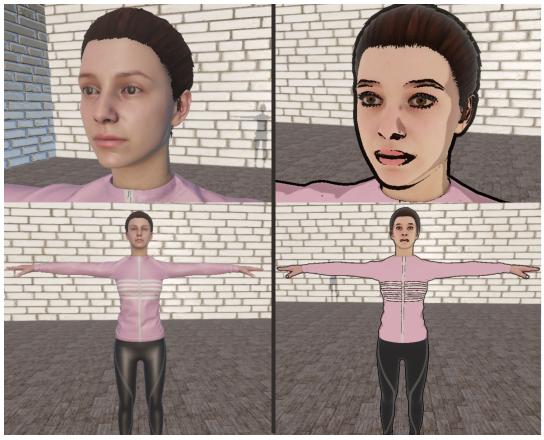

Figure 6: Comparison of Realistic Jody and V3

4.4 Version 4: Gaussian Pre-Filtering

The parameter sensitivity observed in Version 3 highlights a fundamental tension in digital image processing. As Marr and Hildreth [11] observed, differentiation is an ill-posed problem. They noted that some form of low-pass filtering is necessary to achieve stable edge detection because the mathematical act of taking a derivative inherently amplifies components at high frequencies.

Without a pre-processing step to smooth the texture, the shader emphasizes spurious gradients that a human observer would intuitively discount. By incorporating a Gaussian pre-filter, the shader can suppress artifacts while preserving the meaningful intensity transitions that define the character's form. This leads to a more stable and aesthetically consistent result.

4.4.1 Gaussian Pre-Filtering: A Classical Solution. The standard solution is to smooth the image before computing derivatives for the Sobel edge detection. Canny's influential paper on edge detection [4] explicitly recommends this approach, demonstrating that the first stage of edge detection is almost always some form of smoothing, typically low-pass filtering of the image. By attenuating high-frequency noise prior to differentiation, the smoothing step ensures that only genuine structural boundaries produce strong gradient responses.

The Gaussian filter is the preferred choice because it smooths equally in all directions without introducing orientation bias [11]. Practically, the Gaussian provides optimal localization, effectively quieting high-frequency texture noise while maintaining the spatial integrity of meaningful edge positions. Since differentiation and convolution are linear operators, the combined operation of Gaussian smoothing followed by gradient computation can be expressed as:

$$\nabla(G_\sigma * I) = (\nabla G_\sigma) * I \tag{16}$$

where G_σ represents the Gaussian kernel with standard deviation σ, I is the input image, and $*$ denotes convolution.

4.4.2 Implementation. The final version retains the foundational toon shading pipeline and inverted hull silhouette outlines established in earlier versions. The decision to use inverted hull outlines rather than screen-space normal edge detection for silhouettes was motivated by robustness concerns: screen-space derivative methods exhibit distance dependent behavior where edge thickness varies with camera distance.

- **Configurable 9-Tap Gaussian Approximation** This shader version implements a practical approximation of Gaussian pre-filtering using a 9-tap sampling pattern. Rather than performing a true two-pass separable convolution, which would require additional render targets in a single-pass shader, the implementation samples the texture at nine positions arranged in a 3 × 3 grid and computes a weighted average. Crucially, because the entire pipeline executes within a single rendering pass without intermediate render targets, the blurred texture cannot be computed once and stored for the Sobel operator to read from. Instead, the Gaussian blur must be recomputed independently at each of the nine sample positions that the Sobel operator requires. Each such computation involves its own nine-tap Gaussian neighborhood, resulting in a total of 9 × 9 = 81 texture samples per fragment. This represents a significant increase over the 9 samples required by the unfiltered Sobel operator in Version 3, and constitutes the primary performance trade-off of the pre-filtering approach.
  The sampling positions correspond to:
  - 1 center sample
  - 4 cardinal samples (up, down, left, right)
  - 4 diagonal samples (corners)

  The weights are chosen to approximate a Gaussian distribution. For a discrete 3×3 approximation of the Gaussian kernel, the standard weights are:

$$K = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \tag{17}$$

  This corresponds to center weight $\frac{4}{16} = 0.25$, cardinal weights $\frac{2}{16} = 0.125$, and diagonal weights $\frac{1}{16} = 0.0625$
  Unlike fixed parameter implementations, Version 4 exposes adjustable parameters for the Gaussian weights. This design choice reflects the observation that different textures and styles may benefit from different amounts of pre-filtering. The adjustable parameters are:
  - **Center Weight**: Controls the contribution of the unshifted sample. Higher values preserve more original detail; lower values increase smoothing. Default: 0.25.
  - **Cardinal Weight**: Controls the contribution of the four orthogonal neighbours. These samples have the strongest smoothing effect per unit weight due to their proximity. Default: 0.125.
  - **Diagonal Weight**: Controls the contribution of the four corner samples. Due to their greater distance ($\sqrt{2}$ times the cardinal offset), diagonals have less influence in a true Gaussian distribution. Default: 0.0625.
  - **Blur Radius Multiplier**: Scales the sampling offset, effectively controlling the spatial extent of the blur kernel without changing the weights.
- **Post-Blur Edge Refinement**

After Gaussian pre-filtering, the Sobel operator produces gradient magnitudes that are smoother but potentially less sharp than desired for the cartoon aesthetic. Version 4 addresses this through a series of optional post-processing steps.

- **Progressive Smoothstep Passes**
  To sharpen the thresholded edges into the crisp lines characteristic of hand-drawn animation, this shader variant applies up to four successive smoothstep passes. According to Ebert, this technique is analogous to sigmoid sharpening used in neural network training and produces clean, well-defined edges without introducing ringing artifacts [5]. Each pass uses the same S-curve transformation but with progressively wider transition windows.
  - **Pass 1** ($h = 0.03$): Tightest window, aggressively pushes values toward binary
  - **Pass 2** ($h = 0.15$): Moderate sharpening, refines intermediate values
  - **Pass 3** ($h = 0.25$): Wider transition, smooths remaining gradients
  - **Pass 4** ($h = 0.35$): Widest window, final cleanup pass

  Repeated application of smoothstep progressively sharpens the transition, pushing intermediate values toward 0 or 1 while making the edge detection sharper on blurred image.

- **Power Curve** A final optional power curve allows further adjustment of edge contrast:

$$e' = e^{\gamma} \tag{18}$$

where γ is the power exponent. Values greater than 1 suppress weak edges while preserving strong ones; values less than 1 boost weak edges. This provides a simple gamma-style control for overall edge visibility.

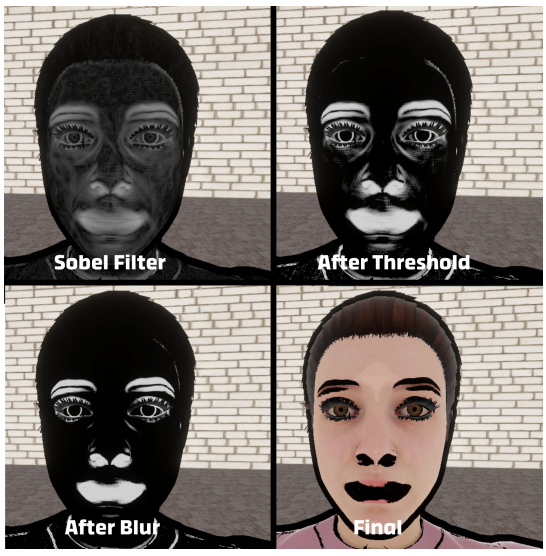

Figure 7: Stages of Implementations

4.4.3 Observations. Testing of the final shader iteration revealed limited perceptual improvement. Whilst the Gaussian pre-filter attenuated spurious detections at lower thresholds, it simultaneously softened meaningful edges, producing smoother but not more selective inner detail lines. Careful parameter tuning across all available controls did not yield a perceptually significant improvement relative to an optimized Version 3 configuration. These trade-offs are analyzed further in Section 5.1.1.

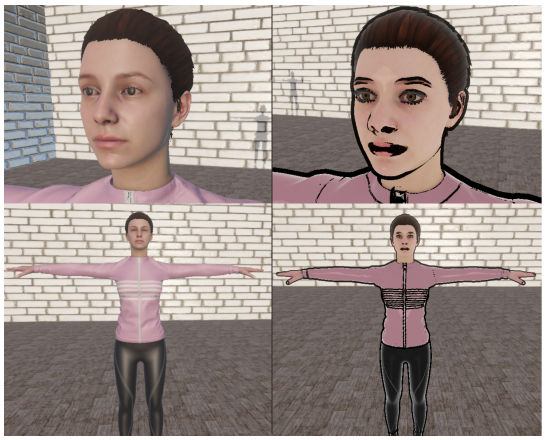

Figure 8: Comparison of Realistic Jody and V4

5 Discussion

The results of this iterative development process suggested that achieving a cohesive non-photorealistic representation of the 'Jody' avatar benefits from a multi-layered approach that balances geometric structure with image-space detail. While initial versions focused on silhouette definition through inverted hull geometry, the subsequent integration of texture-based Sobel filters was essential for preserving expressive features like facial contours and clothing folds.

5.1 Comparison of Edge Detection Approaches

The four shader versions implemented in this work represent fundamentally different approaches to edge detection, each operating in a distinct coordinate space with corresponding advantages and limitations.

Table 1: Edge Detection Space Comparison

Technique	Space	DI	Detects
Geometry Outline	World	Yes	Silhouettes only
Normal Derivatives	Screen	No	Geometry edges
Sobel (Raw)	UV/Texture	Partial	Texture edges
Sobel (Filtered)	UV/Texture	Partial	Texture edges

Note: DI denotes distance independence, indicating whether the detected edges remain visually consistent regardless of camera distance.

5.1.1 Gaussian Pre-Filtering: Mixed Results. The introduction of Gaussian pre-filtering in Version 4 was motivated by Canny's observation that smoothing before differentiation suppresses noise while preserving structural edges. However, testing revealed that this theoretical advantage did not translate directly into practical improvements for avatar rendering.

The Gaussian blur successfully attenuated spurious edge detections at lower threshold values as the artifacts that introduced noticeable instability in Version 3 were compressed. Despite this, the smoothing operation proved fundamentally indiscriminate: it reduced gradient magnitudes for both noise and genuine structural boundaries. The result was smoother inner detail lines rather than more selective edge detection.

Table 2: Gaussian Pre-Filtering Effects

Aspect	Improvement	Trade-off
Artifact presence	Reduced	Meaningful edges also softened
Parameter stability	Moderate improvement	Still requires careful tuning
Edge sharpness	Adjustable	Edges appear smoother
Edge localization	Adjustable	Reduced precision
Configurability	High	More parameters to tune

Careful parameter tuning across the available controls (Gaussian weights, blur radius, threshold window, smoothstep tightness, power curve) did not yield significantly improved edge detection compared to a well-tuned Version 3 configuration. This finding suggests that for avatar rendering, the choice between filtered and unfiltered Sobel depends on aesthetic preference (smoother vs sharper edges) rather than objective quality improvement.

Figure 9: Comparison of Gaussian Blur Parameters

5.2 Implications for VR Avatar Rendering

5.2.1 Silhouette Rendering. The experimental results strongly support world-space inverted hull outlines for VR silhouette rendering. The technique delivers consistent outline width across viewing distances, requires no pre-processing, and exhibits predictable performance characteristics across different avatar topologies. Screen-space alternatives, by contrast, proved unsuitable in contexts involving variable camera distances.

5.2.2 Inner Detail Lines. For internal detail lines such as facial features and clothing folds, texture-based Sobel detection proved the most robust approach within the tested conditions. Geometry outlines are inherently limited to structural silhouettes and cannot respond to texture-painted features, while normal derivatives produced the distance dependent artifacts documented in Section 4.2. Sobel detection, operating directly on texture content, was the only method capable of capturing this layer of visual detail. Nevertheless, threshold tuning requires considerable attention for each avatar region, as settings optimal for one area such as clothing may produce unacceptable results in perceptually prominent areas such as the face.

Table 3: Computational Cost Comparison

Version	Texture Samples	Passes	Notes
V1	1	2	Minimal overhead
V2	1 + ddx/ddy	1	Hardware derivatives free
V3	9	2	Moderate overhead
V4	81 (9×9)	2	Significant overhead

5.2.3 Performance Considerations. Version 4's 81 texture samples per fragment represents a significant computational cost compared to Version 3's 9 samples. Given that Version 4 did not demonstrate clear quality advantages over well-tuned Version 3, the additional cost may not be justified for performance constrained VR applications. Informal testing on VR hardware confirmed that all four shader versions maintained acceptable real-time performance, though systematic GPU profiling was not conducted.

6 Limitations and Future Work

The findings of this work should be interpreted in light of several methodological constraints. Whilst the iterative development process yielded meaningful observations about each shader technique, the evaluation scope was limited by practical considerations of time, hardware access, and the experimental nature of the project.

6.1 Single Avatar Model and Mesh Dependency

All experiments were conducted on the Jody model from Adobe Mixamo. Its specific mesh topology, UV layout, and texture composition inevitably influenced the results. In particular, the parameter sensitivity observed in the Sobel-based edge detection (Version 3) is likely to vary significantly across models with different texture densities or artistic styles. Future work should evaluate the shaders

across a broader set of avatars, including meshes of varying polygon counts and texture styles, to determine whether the findings generalize beyond the specific properties of the Jody model.

6.2 Absence of Quantitative Perceptual Evaluation

The comparative assessment of shader versions throughout this work relied entirely on visual inspection by a single observer rather than quantitative metrics or user studies. Future work should address this through a formal user study employing paired-comparison tasks and standardized instruments such as the Social Presence Questionnaire [13] to assess whether the implemented stylizations reduce discomfort and influence perceived social presence in VR interaction.

6.3 Unexplored Techniques and Temporal Stability

Screen-space normal-edge detection (Version 2) was discontinued after distance artifacts emerged, without exploring mitigation approaches such as depth compensated threshold normalization, which could recover the technique's efficiency advantages. Additionally, only Gaussian pre-filtering was tested as a noise suppression strategy; edge-preserving alternatives such as bilateral filtering may resolve the indiscriminate smoothing observed in Version 4 without attenuating genuine structural boundaries.

7 Conclusion

This work investigated four NPR shader techniques for stylizing a realistic VR avatar, motivated by the need to mitigate the uncanny valley effect while preserving visual expressiveness. No single technique proved sufficient for a complete representation. World-space inverted hull outlines (Version 1) provided temporally stable silhouettes unaffected by camera distance, confirming their suitability as a foundation for VR stylization. Screen-space normal-derivative detection (Version 2) proved unsuitable due to distance dependent edge thickness artifacts. Texture-based Sobel detection (Version 3) successfully introduced inner detail lines but exhibited high threshold sensitivity. Gaussian pre-filtering (Version 4), motivated by classical signal processing recommendations [4, 11], reduced spurious detections but proved indiscriminate: it attenuated both noise and genuine boundaries, yielding smoother rather than more selective lines. Under the tested conditions, this did not translate into a perceptible improvement over a well-tuned Version 3.

These findings supported a hybrid, multi-layered pipeline in which geometry-based methods provide stable outer silhouettes and image-space techniques contribute expressive inner detail. More broadly, the limited perceptual benefit of Gaussian smoothing highlights the necessity of empirical validation when adapting theoretically grounded methods to real-time shader development. The shader implementations developed during this work remain available for further experimentation, providing a foundation for continued investigation into NPR techniques for VR avatar stylization.

References

[1] Adobe Inc. Mixamo: Web-based 3d character rigging and animation service, 2015. URL https://www.mixamo.com/. Accessed: 2026-02-04.

[2] Tomas Akenine-Möller, Eric Haines, Naty Hoffman, Angelo Pesce, Michał Iwanicki, and Sébastien Hillaire. *Real-Time Rendering*. A K Peters/CRC Press, Boca Raton, 4th edition, 2018. ISBN 978-1138627000.

[3] Ryan Canales, Doug Roble, and Michael Neff. The impact of avatar stylization on trust. In *2024 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*, pages 418–428. IEEE, 2024. doi: 10.1109/vr58804.2024.00063.

[4] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6):679–698, 1986. doi: 10.1109/TPAMI.1986.4767851.

[5] David S. Ebert, F. Kenton Musgrave, Darwyn Peachey, Ken Perlin, and Steven Worley. *Texturing & Modeling: A Procedural Approach*. Morgan Kaufmann, San Francisco, 3rd edition, 2003. ISBN 978-1558608481.

[6] James D. Foley, Andries van Dam, Steven K. Feiner, and John F. Hughes. *Computer Graphics: Principles and Practice*. Addison-Wesley, 2nd edition, 1995.

[7] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 4th edition, 2018. ISBN 978-0133356724.

[8] Amy Gooch, Bruce Gooch, Peter Shirley, and Elaine Cohen. A non-photorealistic lighting model for automatic technical illustration. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '98)*, pages 447–452. ACM, 1998. doi: 10.1145/280814.280950.

[9] Tobias Isenberg, Bert Freudenberg, Nick Halper, Stephan Schlechtweg, and Thomas Strothotte. A developer's guide to silhouette algorithms for polygonal models. *IEEE Computer Graphics and Applications*, 23(4):28–37, 2003.

[10] Adam Lake, Carl Marshall, Mark Harris, and Marc Blackstein. Stylized rendering techniques for scalable real-time 3D animation. In *Proceedings of the 1st International Symposium on Non-photorealistic Animation and Rendering (NPAR '00)*, pages 13–20. ACM, 2000. doi: 10.1145/340916.340918.

[11] David Marr and Ellen Hildreth. Theory of edge detection. *Proceedings of the Royal Society of London. Series B, Biological Sciences*, 207(1167):187–217, 1980. doi: 10.1098/rspb.1980.0020.

[12] Masahiro Mori. The uncanny valley. *Energy*, 7(4):33–35, 1970. Translated by K. F. MacDorman and N. Kageki, IEEE Robotics & Automation Magazine, 19(2): 98–100, 2012.

[13] Kristine L. Nowak and Frank Biocca. The effect of the agency and anthropomorphism on users' sense of telepresence, copresence, and social presence in virtual environments. *Presence: Teleoperators and Virtual Environments*, 12(5):481–494, 2003.

[14] Christophe Schlick. An inexpensive brdf model for physically-based rendering. In *Computer Graphics Forum*, volume 13, pages 233–246. Blackwell Publishing Ltd, 1994.

[15] Unity Technologies. Unity real-time development platform, 2023. URL https://unity.com/. Accessed: 2026-02-04.

[16] Unity Technologies. ShaderLab reference. Unity Manual, 2023. URL https://docs.unity3d.com/Manual/SL-Reference.html. Accessed: 2026-02-04.

[17] Unity Technologies. Shaderlab: SubShader reference. Unity Manual, 2023. URL https://docs.unity3d.com/Manual/SL-SubShader.html. Accessed: 2026-02-04.

[18] Unity Technologies. Universal render pipeline (urp) overview, 2023. URL https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@latest/. Accessed: 2026-02-04.

[19] Chris Wyman and Morgan McGuire. Hashed alpha testing. In *Proceedings of the 21st ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games (I3D)*, pages 1–9. ACM, 2017. doi: 10.1145/3023368.3023370.